HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

CAA - Mini-projet

Ransomware post-quantique

Dylan Oliveira Ramos

12-12-2025

1. Description du ransomware

Nous avons un client (ordinateur de la victime) et un serveur (contrôlé par l'attaquant), le serveur possède une paire de clés publique/privée pour l'établissement d'un secret partagé et une pour signer les messages. Les deux clés publiques sont intégrées au ransomware (client). Pour des raisons de simplicité, les deux entités sont exécutées dans le même programme.

Le programme propose les options suivantes :
1. `Encrypt` : pour chiffrer tous les fichiers d'un dossier choisi.
2. `Pay` : pour payer la rançon et pouvoir déchiffrer tous les fichiers.
3. `Decrypt one file` : pour déchiffrer un fichier spécifique et payer une plus petite rançon.
4. `Change password` : pour changer le mot de passe utilisé pour tout déchiffrer.

1.1. Niveau de sécurité choisi

Le ransomware utilise le niveau de sécurité **V**, qui offre une sécurité au moins aussi forte que AES-256.

1.2. Communication entre le client et le serveur

1.2.1. Établissement de la communication sécurisée

Au lancement du programme, le client et le serveur établissent un secret partagé en utilisant l'algorithme **Kyber-1024**. Ce secret partagé est ensuite dérivé avec **HKDF** pour obtenir une clé symétrique utilisée pour chiffrer les communications entre le client et le serveur avec **AES-256-GCM**.

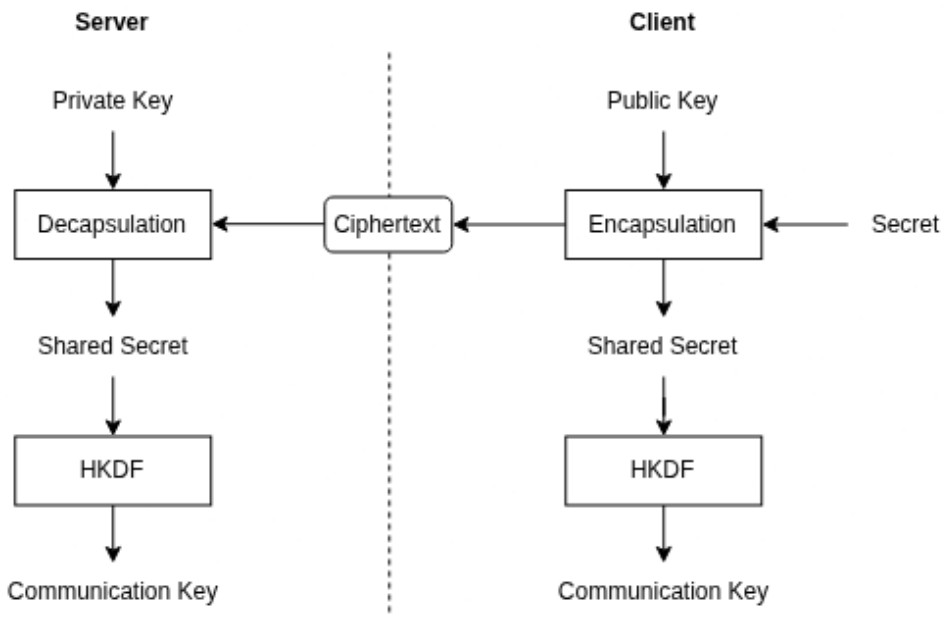

Fig. 1. – Établissement de la clé symétrique pour la communication sécurisée entre le client et le serveur.

1.2.2. Paramètres utilisés

Kyber-1024 :
- Taille de la clé publique : 1568 bytes.
- Taille de la clé privée : 3168 bytes.

HKDF :
- Algorithme de hachage : SHA-256.

- Taille de la clé dérivée : 32 bytes (pour être compatible avec AES-256).
- Sel : aucun.

AES-256-GCM :
- Taille de la clé : 32 bytes.
- Taille du nonce : 12 bytes.
- Taille du tag : 16 bytes.

1.3. Chiffrement des fichiers

1.3.1. Types de clés

Trois types de clés utilisés lors du chiffrement des fichiers :
- `Master Key` : clé dérivée avec **Argon2id** à partir d'un mot de passe aléatoire d'un dictionnaire (`Master Password`).
- `Root Key` : clé générée de manière aléatoire, utilisée pour chiffrer les `File Key` avec **AES-GCM**.
- `File Key` : clé dérivée avec **Argon2id** à partir d'un mot de passe aléatoire du dictionnaire, unique pour chaque fichier, utilisée pour chiffrer le fichier avec **AES-GCM**.

1.3.2. Processus de chiffrement

Lors du choix de l'option `Encrypt`, le client :
1. Chiffre chaque fichier du dossier avec sa `File Key` respective.
2. Chiffre chaque `File Key` avec la `Root Key`.
3. Chiffre la `Root Key` avec la `Master Key`.
4. Stocke la `Root Key` chiffrée et les métadonnées dans un fichier à la racine du dossier.

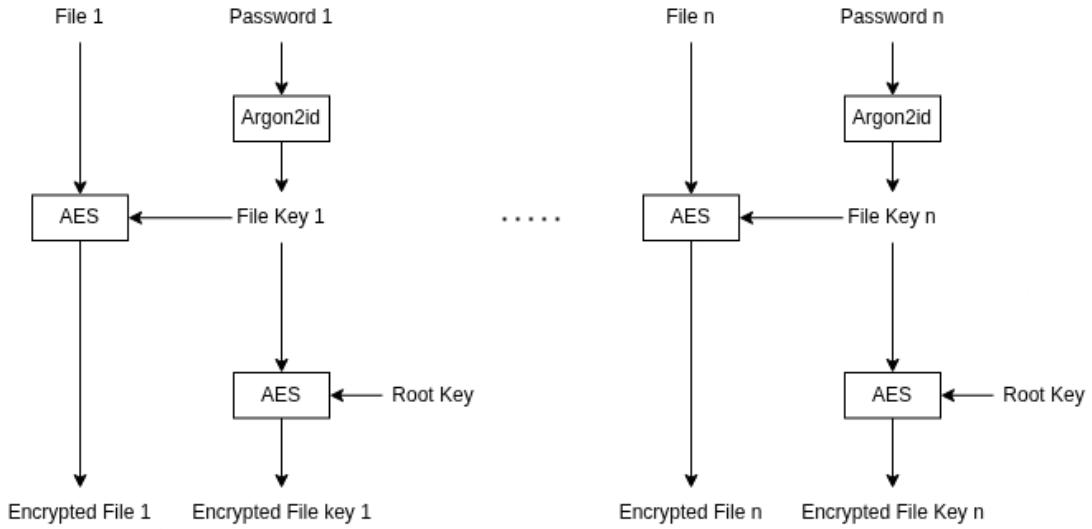

Fig. 2. – Chiffrement des fichiers et des clés de fichier sur le client.

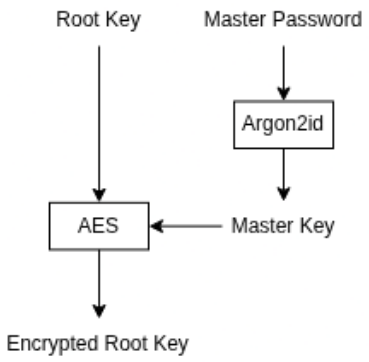

Fig. 3. – Chiffrement de la `Root Key` sur le client.

1.3.3. Paramètres utilisés

Argon2id :

- Taille du sel : 16 bytes.
- Taille de la clé dérivée : 32 bytes (pour être compatible avec AES-256).
- Nombre d'itérations : 1.
- Degré de parallélisme : 4.
- Coût en mémoire : 65536 KB.

AES-256-GCM :

- Taille de la clé : 32 bytes.
- Taille du nonce : 12 bytes.
- Taille du tag : 16 bytes.

Ces paramètres permettent de chiffrer des fichiers d'une taille maximale d'environ 68 GB.

1.3.4. Structure des fichiers chiffrés

Le tableau ci-dessous montre la structure d'un fichier chiffré, chaque donnée est concaténée dans l'ordre indiqué.

File ID	Password Salt	File Key IV	File Key Tag	File Key Ciphertext	File IV	File Tag	File Ciphertext

Le tableau ci-dessous montre la structure du fichier créé à la racine du dossier, chaque donnée est concaténée dans l'ordre indiqué.

File ID	Master Password Salt	Root Key IV	Root Key Tag	Root Key Ciphertext

Note : Le `File ID` 0 est réservé pour le fichier de métadonnées à la racine.

1.3.5. Envoi des mots de passe au serveur

Une fois le chiffrement terminé, le client envoie tous les mots de passes utilisés pour dériver les clés (`Master Password` et `File Passwords`) au serveur en chiffrant le tout avec la `Communication Key`.

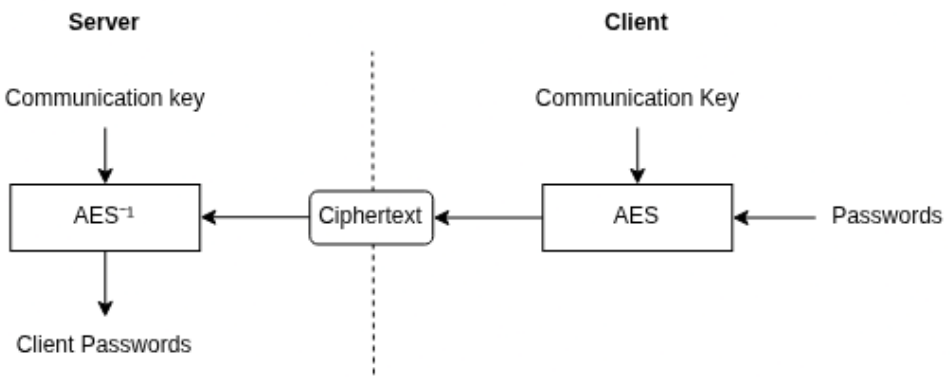

Fig. 4. – Envoi des mots de passe au serveur.

1.4. Paiement de la rançon

1.4.1. Processus de déchiffrement

Lors du choix de l'option `Pay` :

1. Le serveur chiffre le `Master Password` avec la `Communication Key`, signe le tout avec sa clé privée en utilisant l'algorithme **Dilithium 5**, puis l'envoie au client.
2. Le client vérifie la signature avec la clé publique du serveur, puis déchiffre le `Master Password` avec la `Communication Key`.
3. Le client utilise le `Master Password` pour dériver la `Master Key` puis déchiffre la `Root Key`.
4. Le client utilise la `Root Key` pour déchiffrer chaque `File Key`, puis utilise chaque `File Key` pour déchiffrer chaque fichier.

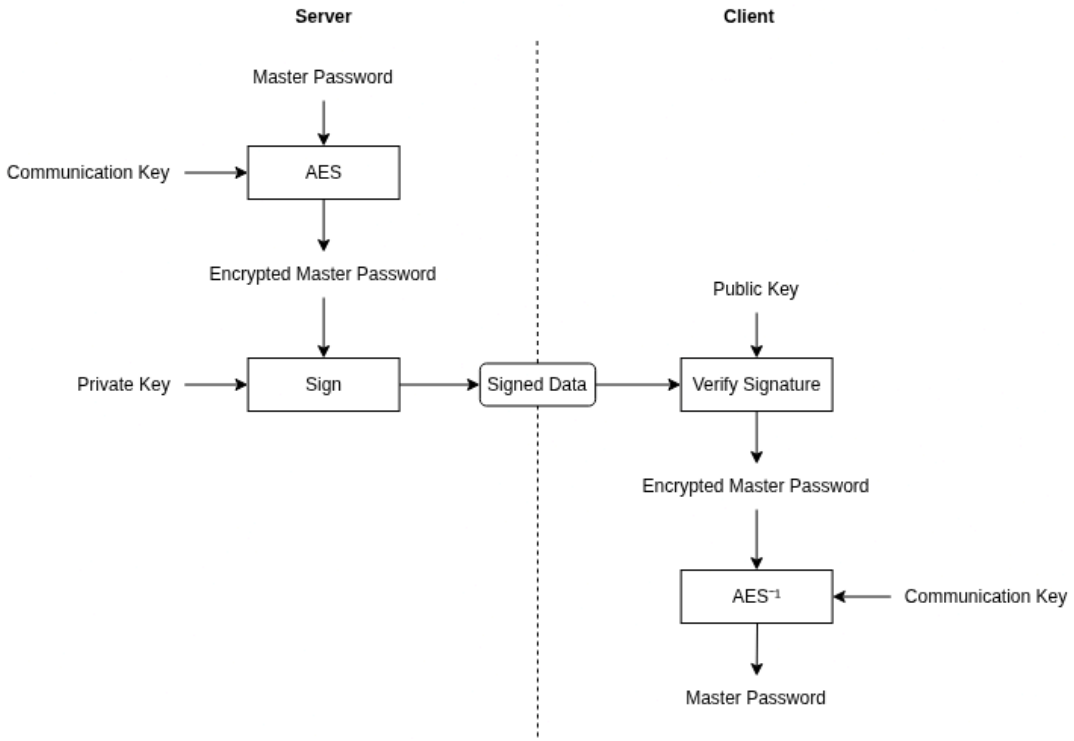

Fig. 5. – Envoi du `Master Password` au client.

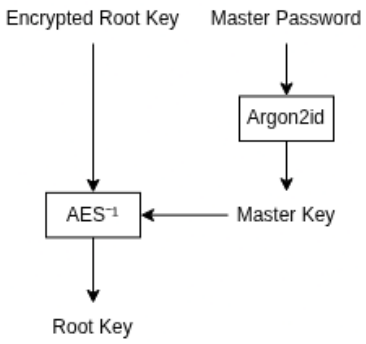

Fig. 6. – Déchiffrement de la `Root Key` sur le client.

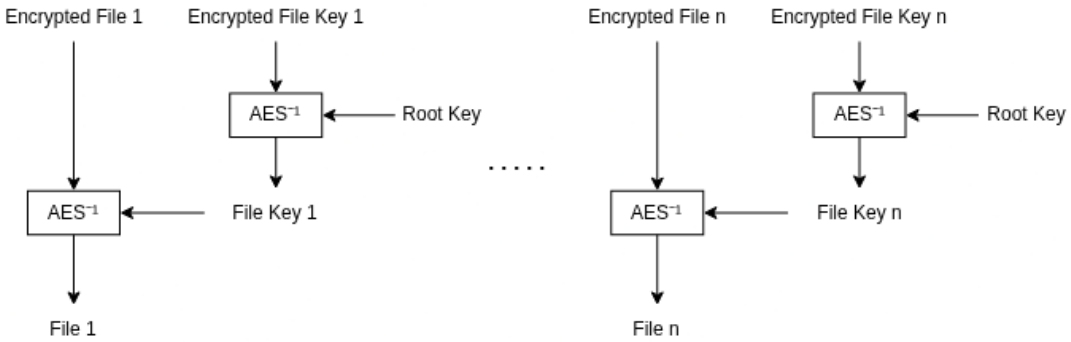

Fig. 7. – Déchiffrement des clés de fichier et des fichiers sur le client.

1.4.2. Paramètres utilisés

Dilithium 5 :

- Taille de la clé publique : 2592 bytes.
- Taille de la clé privée : 4864 bytes.
- Taille de la signature : 4595 bytes.

1.5. Déchiffrement d'un fichier spécifique

Lors du choix de l'option `Decrypt one file` :

1. Le client récupère le `File ID` du fichier, le chiffre avec la `Communication Key`, puis l'envoie au serveur.
2. Le serveur déchiffre le `File ID`, récupère le mot de passe correspondant, le chiffre avec la `Communication Key`, signe le tout avec sa clé privée, puis l'envoie au client.
3. Le client vérifie la signature avec la clé publique du serveur, puis déchiffre le mot de passe avec la `Communication Key`.
4. Le client utilise le mot de passe pour dériver la `File Key`, puis déchiffre le fichier.

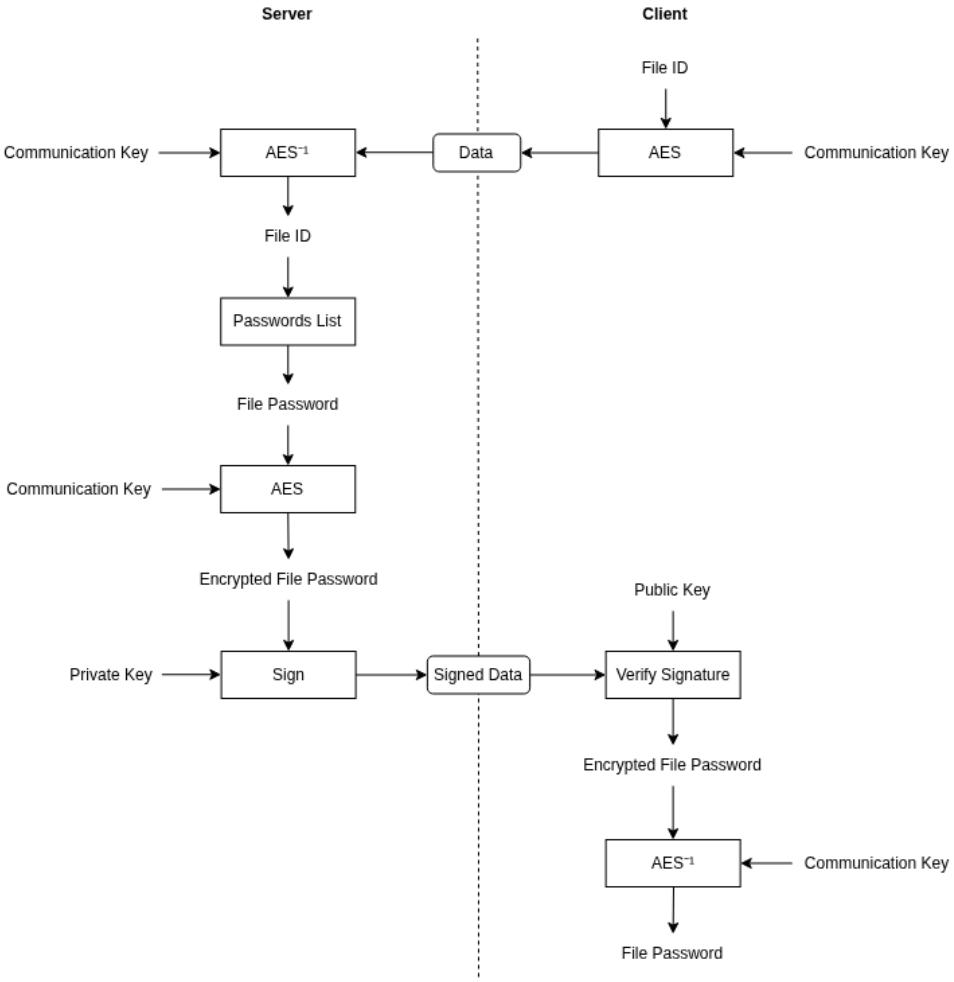

Fig. 8. – Envoi du mot de passe du fichier au client.

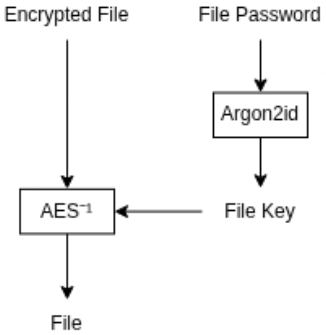

Fig. 9. – Déchiffrement d'un fichier spécifique sur le client.

1.6. Changement de mot de passe

Lors du choix de l'option `Change password` :

1. Le client obtient un nouveau mot de passe aléatoire à partir du dictionnaire, récupère les métadonnées du fichier à la racine du dossier, chiffre le tout avec la `Communication Key`, puis l'envoie au serveur.
2. Le serveur déchiffre les données, dérive la `Master Key` avec l'ancien mot de passe, puis déchiffre la `Root Key`.
3. Le serveur dérive la nouvelle `Master Key` avec le nouveau mot de passe, puis chiffre la `Root Key` avec cette nouvelle clé.

4. Le serveur envoie les nouvelles métadonnées au client en les chiffrant avec la `Communication Key` et en les signant avec sa clé privée.
5. Le serveur met à jour sa liste de mots de passe.
6. Le client vérifie la signature avec la clé publique du serveur, puis déchiffre les nouvelles métadonnées avec la `Communication Key` et les stocke dans le fichier à la racine du dossier.

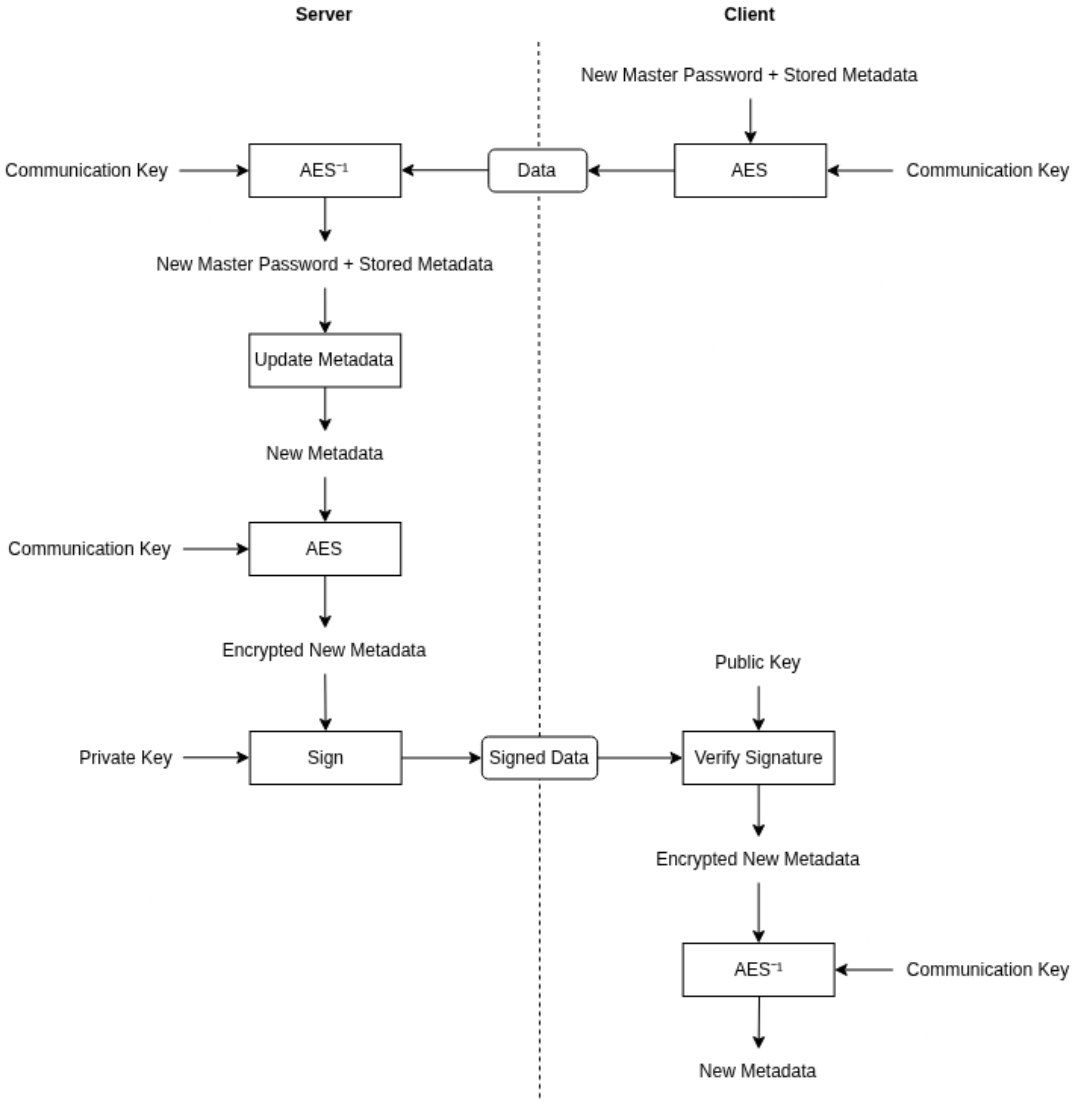

Fig. 10. – Changement du `Master Password` sur le serveur et mise à jour des métadonnées sur le client.

1.7. Spécificités

1.7.1. Résistance aux attaques post-quantiques

Cette architecture est résistante aux attaques post-quantiques car

1.7.2. Pourquoi le niveau de sécurité V est le même partout ?

1.7.3. Qu'est-ce qui permet au ransomware d'être sûr que le mot de passe est légitime ?